

SLOT 13: Cors

Tóm tắt

CORS giúp bảo vệ API khỏi các truy vấn trái phép từ các miền khác, nhưng cần được cấu hình hợp lý để đảm bảo bảo mật và tính linh hoạt khi truy cập dữ liệu từ frontend

- Hiểu CORS là gì và tại sao cần nó.
- Cách trình duyệt xử lý CORS thông qua HTTP headers.
- Cấu hình CORS trong Express bằng `cors` middleware.
- Kiểm tra API hỗ trợ CORS bằng Postman..

CÁC NỘI DUNG TRỌNG TÂM

1. CORS là gì?

- CORS (Cross-Origin Resource Sharing) là một tính năng bảo mật của trình duyệt.
- Nó ngăn chặn các trang web độc hại truy cập tài nguyên từ một miền khác mà không được phép.
- CORS sử dụng các HTTP headers để kiểm soát quyền truy cập giữa các nguồn khác nhau.

2. Cách CORS hoạt động

- Khi trình duyệt gửi yêu cầu đến một miền khác (cross-origin request), nó sẽ kiểm tra xem máy chủ có cho phép truy cập hay không.
- Nếu máy chủ cho phép, trình duyệt sẽ thực hiện yêu cầu.
- Nếu không có tiêu đề cho phép CORS, trình duyệt sẽ chặn yêu cầu.

3. Các HTTP Headers quan trọng trong CORS

- **Access-Control-Allow-Origin:** Xác định các miền được phép truy cập tài nguyên.
- **Access-Control-Allow-Methods:** Xác định các phương thức HTTP được phép (GET, POST, PUT, DELETE,...).
- **Access-Control-Allow-Headers:** Chỉ định các headers có thể được sử dụng trong yêu cầu.
- **Access-Control-Max-Age:** Xác định thời gian trình duyệt có thể lưu cache kết quả kiểm tra CORS (preflight request).

4. Preflight Request trong CORS

- Đối với các yêu cầu phức tạp (như `PUT`, `DELETE`, hoặc khi gửi headers tùy chỉnh), trình duyệt gửi một yêu cầu kiểm tra trước (`OPTIONS`).
- Máy chủ phản hồi với các headers CORS để xác nhận có cho phép yêu cầu chính hay không.
- Nếu máy chủ không trả về các headers phù hợp, trình duyệt sẽ chặn yêu cầu.

5. Cấu hình CORS trong Express

- Cài đặt `cors` middleware trong Node.js:

```
sh
npm install cors
```

- Kích hoạt CORS trên toàn bộ ứng dụng:

```
js
const cors = require('cors');
app.use(cors());
```

- Cấu hình CORS chỉ cho một số miền nhất định:

```
js
app.use(cors({
  origin: 'https://example.com',
  methods: ['GET', 'POST'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

6. Kiểm thử CORS bằng Postman

- Gửi các yêu cầu **GET, POST, PUT, DELETE** để kiểm tra phản hồi của máy chủ.
- Quan sát phản hồi HTTP headers để kiểm tra xem máy chủ có hỗ trợ CORS không.

7. Các vấn đề thường gặp với CORS và cách khắc phục

- **Lỗi CORS Policy:** Máy chủ không trả về `Access-Control-Allow-Origin`.
→ Cần cấu hình CORS đúng cách trên máy chủ.
- **Không thể gửi yêu cầu từ trình duyệt:** Trình duyệt chặn yêu cầu vì không có preflight request hợp lệ.
→ Cần thêm `OPTIONS` handler trong Express.
- **Token không được gửi kèm yêu cầu:** CORS mặc định không gửi credentials (cookies, token).
→ Cần bật `credentials: true`:

```
js
app.use(cors({
  origin: 'https://example.com',
  credentials: true
}));
```

ỨNG DỤNG TRONG THỰC TIỄN

CORS (Cross-Origin Resource Sharing) là một kỹ thuật quan trọng khi phát triển ứng dụng Node.js và RESTful APIs

Ứng dụng	Mô tả	Giải pháp
Tích hợp frontend-backend	Cho phép React/Vue/Angular gọi API Node.js	Cấu hình CORS trong Express
API công khai & bên thứ ba	Cho phép nhiều domain truy cập API	<code>origin: '*'</code> hoặc danh sách cụ thể
Microservices giao tiếp	Cho phép các service trên nhiều miền trao đổi dữ liệu	Cấu hình CORS giữa các microservices
Môi trường phát triển	Hỗ trợ làm việc với frontend và backend local	Chỉ bật CORS khi <code>NODE_ENV=development</code>
Xác thực với token/cookies	Gửi cookies/tokens trong yêu cầu cross-origin	<code>credentials: true</code>
Tích hợp API bên thứ ba	Gọi API từ Google Maps, Stripe, PayPal	Dùng proxy nếu cần
Bảo mật CORS	Ngăn chặn truy cập trái phép vào API	Chỉ cho phép origin tin cậy

1. Tích hợp Frontend và Backend từ các nguồn gốc khác nhau

Tình huống thực tế:

- Một ứng dụng **React, Angular, Vue** chạy trên `http://localhost:3000`.
- API backend của ứng dụng chạy trên `http://localhost:5000`.
- Nếu không bật CORS, frontend sẽ không thể gọi API do trình duyệt chặn yêu cầu.

Giải pháp:

Sử dụng CORS trong Express để cho phép frontend truy cập API:

```
js

const cors = require('cors');
const app = require('express')();

app.use(cors({ origin: 'http://localhost:3000' }));

app.get('/data', (req, res) => {
  res.json({ message: 'CORS enabled' });
});

app.listen(5000, () => console.log('Server running on port 5000'));
```

→ **Kết quả:** React/Vue/Angular frontend có thể gọi API mà không bị lỗi CORS.

2. Xây dựng API công khai hoặc API cho bên thứ ba

Tình huống thực tế:

- Một công ty xây dựng API dịch vụ thời tiết (<https://api.weather.com>) cho phép nhiều website sử dụng.
- Nhiều khách hàng muốn tích hợp API này vào website của họ (<https://example.com>).

Giải pháp:

Bật CORS cho phép tất cả hoặc chỉ các miền cụ thể:

js

```
app.use(cors({ origin: '*' })); // Cho phép tất cả domain
// hoặc chỉ cho phép một số domain cụ thể
app.use(cors({ origin: ['https://example.com', 'https://partner.com'] }));
```

→ **Kết quả:** API có thể được sử dụng bởi nhiều trang web mà vẫn đảm bảo kiểm soát quyền truy cập.

3. Xây dựng Microservices giao tiếp với nhau

Tình huống thực tế:

- Một ứng dụng lớn chia thành nhiều microservices (dịch vụ nhỏ).
- Các dịch vụ chạy trên các domain khác nhau, ví dụ:
 - <https://auth.example.com> (xác thực)
 - <https://api.example.com> (dịch vụ chính)
 - <https://admin.example.com> (bảng điều khiển quản trị)

Giải pháp:

Cấu hình CORS để các microservices có thể giao tiếp mà không bị trình duyệt chặn:

js

```
app.use(cors({ origin: ['https://auth.example.com',
'https://admin.example.com'] }));
```

→ **Kết quả:** Các microservices có thể gửi và nhận dữ liệu mà không bị hạn chế bởi CORS.

4. Hỗ trợ phát triển và kiểm thử ứng dụng trên môi trường Localhost

Tình huống thực tế:

- Backend chạy trên <http://localhost:5000> và frontend chạy trên <http://localhost:3000>.
- Khi gọi API từ frontend, trình duyệt sẽ chặn yêu cầu vì CORS.

Giải pháp:

Cho phép tất cả các yêu cầu từ localhost trong môi trường phát triển:

```
js
if (process.env.NODE_ENV === 'development') {
  app.use(cors({ origin: 'http://localhost:3000' }));
}
```

→ **Kết quả:** Developer có thể làm việc với frontend và backend mà không bị lỗi CORS.

5. Cho phép xác thực với Cookies và Token trong API

Tình huống thực tế:

- Một ứng dụng có **xác thực đăng nhập bằng session/token**.
- CORS mặc định không cho phép gửi **cookies hoặc authentication headers** giữa các domain khác nhau.

Giải pháp:

Bật CORS với tùy chọn `credentials: true`:

```
js
app.use(cors({
  origin: 'https://client.example.com',
  credentials: true
}));
```

→ **Kết quả:** Trình duyệt sẽ gửi **cookies** hoặc **token** theo yêu cầu đến API.

6. Tích hợp API bên thứ ba từ các dịch vụ khác

Tình huống thực tế:

- Một ứng dụng muốn hiển thị bản đồ từ Google Maps API (<https://maps.googleapis.com>).
- Hoặc gọi API thanh toán từ Stripe, PayPal (<https://api.stripe.com>).
- Nếu không có CORS, trình duyệt sẽ chặn các yêu cầu này.

Giải pháp:

Nếu không thể cấu hình CORS trên máy chủ của bên thứ ba, sử dụng **proxy server**:

```
js
const { createProxyMiddleware } = require('http-proxy-middleware');

app.use('/api', createProxyMiddleware({
  target: 'https://api.stripe.com',
  changeOrigin: true
}));
```

→ **Kết quả:** Ứng dụng có thể gửi yêu cầu đến dịch vụ bên thứ ba thông qua proxy.

7. Bảo mật CORS để tránh lỗ hổng bảo mật

Tình huống thực tế:

- Nếu CORS bị cấu hình sai (`origin: '*'`), bất kỳ website nào cũng có thể truy cập API.
- Hacker có thể lợi dụng CORS để thực hiện các cuộc tấn công như **CSRF (Cross-Site Request Forgery)**.

Giải pháp:

Chỉ cho phép các miền đáng tin cậy:

js

```
const allowedOrigins = ['https://trusteddomain.com'];

app.use(cors({
  origin: function (origin, callback) {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('CORS policy violation'));
    }
  }
}));
```

→ **Kết quả:** Bảo vệ API khỏi các yêu cầu trái phép từ miền không xác định.

MỘT SỐ NỘI DUNG CHI TIẾT

1. CORS là gì?

CORS (Cross-Origin Resource Sharing) là một **cơ chế bảo mật** của trình duyệt, giúp kiểm soát cách một trang web có thể yêu cầu tài nguyên từ một miền khác (**cross-origin request**).

Ví dụ về cross-origin request:

- Một trang web chạy trên `https://frontend.com` muốn gọi API từ `https://api.backend.com`.
- Theo chính sách Same-Origin Policy (SOP), trình duyệt sẽ **chặn yêu cầu** nếu không có CORS.
- Nếu máy chủ `https://api.backend.com` bật CORS, trình duyệt sẽ cho phép truy cập.

Tại sao cần CORS?

- **Bảo vệ dữ liệu người dùng:** Ngăn chặn các trang web độc hại truy cập API trái phép.
- **Hỗ trợ tích hợp hệ thống:** Cho phép giao tiếp giữa frontend và backend khác miền.
- **Hỗ trợ microservices:** Các dịch vụ chạy trên nhiều miền khác nhau có thể giao tiếp với nhau.

2. Cách hoạt động của CORS

Quy trình xử lý CORS:

1. Trình duyệt gửi yêu cầu đến máy chủ kèm theo Origin Header:

```
http
GET /data HTTP/1.1
Host: api.backend.com
Origin: https://frontend.com
```

2. Máy chủ phản hồi với CORS Header nếu cho phép:

```
http
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://frontend.com
```

3. Trình duyệt kiểm tra phản hồi, nếu hợp lệ thì cho phép truy cập.

Nếu máy chủ không bật CORS:

- Trình duyệt sẽ chặn yêu cầu và hiển thị lỗi **CORS Policy Blocked** trên console.

3. Các HTTP Headers quan trọng trong CORS

1. Request Headers (do trình duyệt gửi lên máy chủ)

Header	Ý nghĩa
Origin	Xác định nguồn gốc (domain) của yêu cầu

2. Response Headers (do máy chủ gửi về trình duyệt)

Header	Ý nghĩa
Access-Control-Allow-Origin	Xác định miền nào được phép truy cập (hoặc * cho phép tất cả)
Access-Control-Allow-Methods	Xác định các HTTP methods được phép (GET, POST, PUT, DELETE, ...)
Access-Control-Allow-Headers	Xác định các headers mà client có thể gửi đi
Access-Control-Allow-Credentials	Cho phép gửi cookies hoặc authentication tokens
Access-Control-Max-Age	Xác định thời gian cache kết quả preflight request

4. Preflight Request trong CORS

Preflight Request là một yêu cầu kiểm tra trước khi gửi **yêu cầu chính**. Nó giúp đảm bảo rằng máy chủ cho phép truy cập.

◆ Khi nào cần Preflight Request?

- Khi sử dụng **phương thức HTTP không chuẩn** (POST, PUT, DELETE, OPTIONS).
- Khi gửi yêu cầu có **headers đặc biệt** (ví dụ: Authorization, Content-Type khác application/x-www-form-urlencoded).

◆ Ví dụ về Preflight Request:

1. Trình duyệt gửi yêu cầu OPTIONS trước khi gửi yêu cầu chính:

```
http
OPTIONS /data HTTP/1.1
Origin: https://frontend.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type
```

2. Máy chủ phản hồi cho phép hoặc từ chối:

```
http
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://frontend.com
Access-Control-Allow-Methods: POST, GET
Access-Control-Allow-Headers: Content-Type
```

3. Nếu được cho phép, trình duyệt sẽ tiếp tục gửi yêu cầu chính.

5. Cấu hình CORS trong Express.js

Cài đặt thư viện CORS

```
sh
npm install cors
```


Bật CORS cho toàn bộ ứng dụng

js

```
const cors = require('cors');
const express = require('express');
const app = express();

app.use(cors()); // Cho phép tất cả origin

app.get('/data', (req, res) => {
  res.json({ message: 'CORS enabled' });
});

app.listen(5000, () => console.log('Server running on port 5000'));
```

👉 Khi dùng `app.use(cors())`, API có thể được truy cập từ bất kỳ domain nào (Access-Control-Allow-Origin: *).

6. Cấu hình CORS nâng cao

Chỉ cho phép một số domain cụ thể

js

```
const corsOptions = {
  origin: ['https://frontend.com', 'https://trusted.com'],
  methods: ['GET', 'POST'],
  allowedHeaders: ['Content-Type', 'Authorization']
};

app.use(cors(corsOptions));
```

◆ Giải thích:

- Chỉ chấp nhận yêu cầu từ `https://frontend.com` và `https://trusted.com`.
- Chỉ cho phép các phương thức GET và POST.
- Chỉ chấp nhận Content-Type và Authorization trong headers.

Cho phép gửi credentials (cookies, tokens)

◆ Mặc định, CORS không cho phép gửi credentials.

◆ Để bật tính năng này, cần cấu hình cả frontend và backend:

Backend (Node.js)

js

```
app.use(cors({
  origin: 'https://frontend.com',
  credentials: true
}));
```

Frontend (React, Angular, Vue)

js

```
fetch('https://api.backend.com/data', {  
  credentials: 'include'  
});
```

👉 **Giúp client gửi cookies/session khi gọi API.**

7. Kiểm tra CORS bằng Postman

Các bước kiểm tra:

1. Mở **Postman**.
2. Gửi các yêu cầu **GET, POST, PUT, DELETE** đến API.
3. Quan sát phản hồi và kiểm tra **CORS headers**.

8. Các lỗi CORS phổ biến và cách khắc phục

Lỗi	Nguyên nhân	Cách khắc phục
CORS Policy Blocked	Máy chủ không bật CORS	Cấu hình <code>cors()</code> trong Express
Preflight Request Failed	Máy chủ không xử lý OPTIONS	Thêm handler cho OPTIONS request
Credentials mode is 'include' but Access-Control-Allow-Credentials is missing	Không có <code>Access-Control-Allow-Credentials: true</code>	Bật <code>credentials: true</code> trong CORS
Access-Control-Allow-Origin: * và credentials cùng tồn tại	Không thể dùng * với <code>credentials: true</code>	Chỉ định cụ thể origin thay vì *

9. Ứng dụng thực tế của CORS

- Giao tiếp giữa frontend (React, Vue, Angular) và backend (Node.js, Express).
- Tích hợp API bên thứ ba như Google Maps, Stripe, PayPal.
- Cho phép nhiều domain truy cập một API công khai.
- Giao tiếp giữa các microservices trong một hệ thống phân tán.

KINH NGHIỆM THỰC TIỄN

1. Hạn chế sử dụng `Access-Control-Allow-Origin: *` trong môi trường Production

◆ Vấn đề:

- Khi sử dụng `Access-Control-Allow-Origin: *`, API sẽ cho phép **bất kỳ trang web** nào truy cập tài nguyên của bạn.
- Điều này có thể gây rủi ro bảo mật, đặc biệt nếu API có chứa dữ liệu nhạy cảm hoặc yêu cầu xác thực.

✅ Giải pháp:

- Chỉ cho phép các **origin** đáng tin cậy.
- Cấu hình `cors` trong Express chỉ cho phép một số domain cụ thể:

```
js

const allowedOrigins = ['https://frontend.com', 'https://trusted.com'];

app.use(cors({
  origin: function (origin, callback) {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('CORS policy violation'));
    }
  }
}));
```

◆ Lợi ích:

- ✅ Ngăn chặn truy cập trái phép từ các nguồn không xác định.
- ✅ Bảo mật API khỏi các cuộc tấn công từ bên ngoài.

2. Xử lý lỗi CORS do Preflight Request

◆ Vấn đề:

- Một số yêu cầu `POST`, `PUT`, `DELETE` hoặc có headers đặc biệt (như `Authorization`) sẽ kích hoạt **Preflight Request** (`OPTIONS`).
- Nếu backend không xử lý yêu cầu này, trình duyệt sẽ chặn truy cập.

✅ Giải pháp:

- Cấu hình máy chủ Express để xử lý **OPTIONS request**:

```
js

app.options('*', cors()); // Cho phép mọi preflight request
```

- Hoặc cấu hình cụ thể hơn:

```
js

app.use(cors({
  origin: 'https://frontend.com',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

◆ Lợi ích:

- ✓ Giúp API có thể xử lý các yêu cầu phức tạp từ frontend.
- ✓ Tránh lỗi Preflight Request Failed khi gọi API.

3. Cho phép gửi Credentials một cách an toàn

◆ Vấn đề:

- Nếu frontend sử dụng **cookies** hoặc **authentication tokens**, trình duyệt sẽ **không gửi** credentials theo mặc định.
- CORS cần được bật `credentials` để gửi **cookies/token** từ client.

✓ Giải pháp:

- Cấu hình backend:

```
js

app.use(cors({
  origin: 'https://frontend.com',
  credentials: true
}));
```

- Cấu hình frontend (React, Vue, Angular):

```
js

fetch('https://api.backend.com/data', {
  credentials: 'include'
});
```

◆ Lưu ý:

- Khi bật `credentials: true`, **không thể sử dụng `Access-Control-Allow-Origin: *`**.
 ✓ Chỉ định một domain cụ thể thay vì *.

4. Kiểm tra CORS trong môi trường phát triển (Development) và Production

◆ Vấn đề:

- Khi phát triển, frontend thường chạy trên `localhost:3000` trong khi backend chạy trên `localhost:5000`, dẫn đến lỗi CORS.
- API có thể hoạt động trên máy local nhưng gặp lỗi khi triển khai lên Production.

✅ Giải pháp:

- Chỉ bật **CORS rộng rãi** trong môi trường phát triển:

```
js

if (process.env.NODE_ENV === 'development') {
  app.use(cors());
} else {
  app.use(cors({ origin: 'https://frontend.com' }));
}
```

◆ Lợi ích:

- ✅ Tránh lỗi CORS khi kiểm thử trên local.
- ✅ Giữ API bảo mật khi triển khai thực tế.

5. Hạn chế nguy cơ tấn công bằng CORS misconfiguration

◆ Vấn đề:

- Nếu không cấu hình CORS đúng cách, API có thể bị **CSRF (Cross-Site Request Forgery)** hoặc **data leakage**.
- Ví dụ: Nếu hacker tạo một trang giả mạo và gửi request đến API, API có thể **vô tình trả dữ liệu** cho trang giả mạo đó.

✅ Giải pháp:

- Chỉ cho phép các **origin tin cậy**.
- Sử dụng **CSRF tokens** để kiểm tra yêu cầu hợp lệ.
- Kiểm tra **Referer** hoặc **Origin** headers để ngăn request giả mạo:

```
js

app.use((req, res, next) => {
  const allowedOrigins = ['https://frontend.com'];
  if (!allowedOrigins.includes(req.headers.origin)) {
    return res.status(403).json({ error: 'Forbidden' });
  }
  next();
});
```

◆ Lợi ích:

- ✅ Ngăn chặn **lỗ hổng bảo mật** do CORS misconfiguration.
- ✅ Đảm bảo chỉ có client hợp lệ mới có thể gọi API.

6. Sử dụng Proxy Server để tránh lỗi CORS khi không thể thay đổi Backend

◆ Vấn đề:

- Một số API bên thứ ba (Google Maps, PayPal, Stripe) **không bật CORS**, khiến frontend không thể gọi API trực tiếp.

✅ Giải pháp:

- Dùng **proxy server** trên backend để chuyển tiếp yêu cầu.

Ví dụ:

1. Cài đặt `http-proxy-middleware` trong Express:

```
sh  
  
npm install http-proxy-middleware
```

2. Thêm proxy vào backend:

```
js  
  
const { createProxyMiddleware } = require('http-proxy-middleware');  
app.use('/api', createProxyMiddleware({  
  target: 'https://api.stripe.com',  
  changeOrigin: true  
}));
```

◆ Lợi ích:

- ✅ Giúp frontend gọi API mà không bị lỗi CORS.
- ✅ Không cần chỉnh sửa cấu hình trên server bên thứ ba.

7. Kiểm thử CORS trước khi triển khai

◆ Vấn đề:

- CORS có thể hoạt động trên môi trường local nhưng gặp lỗi trên Production do chính sách bảo mật của server.

✅ Giải pháp:

- Kiểm tra CORS bằng **Postman** hoặc **cURL**:

```
sh  
  
curl -H "Origin: https://frontend.com" -X GET  
https://api.backend.com/data -v
```

- Kiểm tra console của trình duyệt để xem lỗi CORS:

sh

```
Access to fetch at 'https://api.backend.com' from origin  
'https://frontend.com' has been blocked by CORS policy
```

◆ Lợi ích:

- ✓ Phát hiện lỗi CORS trước khi triển khai chính thức.
- ✓ Đảm bảo API hoạt động đúng với các miền được cấp quyền.

Kết luận

- ✓ **CORS là một công cụ quan trọng** trong phát triển API, nhưng cần cấu hình **đúng cách** để tránh lỗ hổng bảo mật.
- ✓ **Không sử dụng Access-Control-Allow-Origin: *** trong môi trường **production**.
- ✓ **Luôn kiểm tra CORS trước khi triển khai** để tránh lỗi ngoài mong đợi.
- ✓ **Cân bằng giữa bảo mật và khả năng tương thích** để đảm bảo API hoạt động hiệu quả.

EXERCISE

CORS API Demo

1 Thông Tin Ứng Dụng

- 💡 **Mô tả:** Ứng dụng Node.js sử dụng Express để xây dựng API hỗ trợ CORS, giúp frontend (React, Vue, Angular) có thể gọi API từ domain khác.
- 🎯 **Mục tiêu:**
 - Cấu hình CORS trong Express.
 - Xử lý preflight requests (`OPTIONS`).
 - Hỗ trợ gửi credentials (cookies, tokens).
 - Kiểm tra lỗi và khắc phục lỗi CORS.

2 Chức Năng Chính

- ✅ **Tạo API backend với Express.**
- ✅ **Bật CORS** để hỗ trợ truy cập từ nhiều nguồn khác nhau.
- ✅ **Giới hạn origin truy cập API** (bảo mật).
- ✅ **Hỗ trợ credentials** (cookies, tokens).
- ✅ **Xử lý preflight requests** cho các yêu cầu `POST`, `PUT`, `DELETE`.
- ✅ **Kiểm thử API bằng Postman và trình duyệt.**

3 Công Nghệ Sử Dụng

◆ Backend:

- `Node.js`: Chạy server.
- `Express.js`: Xây dựng API.
- `CORS Middleware`: Hỗ trợ CORS.
- `dotenv`: Quản lý biến môi trường.

◆ Công Cụ Kiểm Thử:

- `Postman`: Kiểm tra API.
- `cURL`: Gửi request từ terminal.

4 Tóm Tắt Các Bước Triển Khai

- ◆ **Bước 1:** Cài đặt môi trường Node.js.
- ◆ **Bước 2:** Tạo API Express và bật CORS.
- ◆ **Bước 3:** Kiểm thử API với Postman.
- ◆ **Bước 4:** Cấu hình CORS nâng cao (cho phép origin cụ thể, credentials, preflight).

- ◆ **Bước 5:** Dùng Proxy Server khi backend không hỗ trợ CORS.
- ◆ **Bước 6:** Xử lý và khắc phục lỗi CORS thường gặp.

5 Chi Tiết Các Bước Triển Khai

🚀 Bước 1: Cài Đặt Môi Trường Node.js

1 Khởi tạo dự án Node.js

```
sh

mkdir cors-api-demo
cd cors-api-demo
npm init -y
```

2 Cài đặt thư viện cần thiết

```
sh

npm install express cors dotenv
```

🚀 Bước 2: Tạo API Express Và Kích Hoạt CORS

1 Tạo file `server.js`

2 Thêm mã nguồn sau vào `server.js`:

```
js

require('dotenv').config();
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = process.env.PORT || 5000;

// Kích hoạt CORS cho tất cả origin
app.use(cors());

// Middleware xử lý JSON
app.use(express.json());

// API đơn giản
app.get('/api/data', (req, res) => {
  res.json({ message: 'CORS is enabled!' });
});

// Chạy server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

3 Chạy ứng dụng:

```
sh
```

```
node server.js
```

Bước 3: Kiểm Thử CORS Bằng Postman

1 **Mở trình duyệt hoặc Postman**, gửi request đến:

```
bash
```

```
http://localhost:5000/api/data
```

2 **Thêm header:**

```
arduino
```

```
Origin: https://another-site.com
```

3 **Kiểm tra phản hồi và header `Access-Control-Allow-Origin`.**

Bước 4: Cấu Hình CORS Nâng Cao

Chỉ Cho Phép Một Số Origin Cụ Thể

```
js
```

```
const corsOptions = {  
  origin: ['http://localhost:3000', 'https://trusted.com'],  
  methods: ['GET', 'POST'],  
  allowedHeaders: ['Content-Type', 'Authorization']  
};
```

```
app.use(cors(corsOptions));
```

Hỗ Trợ Credentials (Cookies, Token Xác Thực) Backend (Node.js)

```
js
```

```
app.use(cors({  
  origin: 'https://frontend.com',  
  credentials: true  
}));
```

Frontend (React, Vue, Angular)

```
js
```

```
fetch('https://api.backend.com/data', {  
  credentials: 'include'  
});
```

Xử Lý Preflight Request (OPTIONS)

```
js
app.options('*', cors()); // Cho phép tất cả preflight requests
```

Bước 5: Dùng Proxy Server Nếu Backend Không Hỗ Trợ CORS

1 Cài đặt http-proxy-middleware:

```
sh
npm install http-proxy-middleware
```

2 Thêm proxy vào backend:

```
js
const { createProxyMiddleware } = require('http-proxy-middleware');

app.use('/proxy-api', createProxyMiddleware({
  target: 'https://api.thirdparty.com',
  changeOrigin: true
}));
```

3 Dùng proxy trong React:

```
js
proxy: {
  '/api': {
    target: 'http://localhost:5000',
    changeOrigin: true,
    secure: false
  }
}
```

Bước 6: Xử Lý Và Khắc Phục Các Lỗi CORS

Lỗi	Nguyên nhân	Cách khắc phục
CORS Policy Blocked	API chưa bật CORS	Thêm <code>app.use(cors())</code>
Preflight Request Failed	Không xử lý OPTIONS	Thêm <code>app.options('*', cors())</code>
Credentials mode is 'include' but Access-Control-Allow-Credentials is missing	Thiếu <code>credentials: true</code>	Bật <code>credentials: true</code> trong CORS
Access-Control-Allow-Origin: * và credentials: true cùng tồn tại	Không thể dùng * với credentials	Chỉ định cụ thể origin thay vì *

CORS Demo App 2

TÓM TẮT:

- **Chức năng:** Đây là một ứng dụng NodeJS đơn giản cho phép client (ví dụ: trình duyệt) từ một nguồn gốc khác (cross-origin) gửi yêu cầu HTTP tới server và nhận phản hồi. Ứng dụng sẽ cung cấp một API cơ bản để trả về dữ liệu JSON.
- **Công nghệ sử dụng:**
 - **NodeJS:** Môi trường chạy JavaScript phía server.
 - **Express:** Framework để xây dựng API.
 - **CORS:** Middleware để xử lý Cross-Origin Resource Sharing.
 - **npm:** Trình quản lý gói để cài đặt các dependencies.

Tổng Quan Các Bước

1. Thiết lập môi trường và khởi tạo dự án NodeJS.
2. Cài đặt các gói cần thiết (Express, CORS).
3. Tạo cấu trúc thư mục và file cơ bản.
4. Xây dựng server với Express và tích hợp CORS.
5. Tạo các endpoint API đơn giản.
6. Kiểm tra ứng dụng với công cụ như Postman hoặc trình duyệt.

Chi Tiết Thực Hiện Các Bước

Bước 1: Thiết Lập Môi Trường và Khởi Tạo Dự án

1. **Cài đặt NodeJS** (nếu chưa có):
 - Truy cập nodejs.org và tải phiên bản LTS (Long Term Support).
 - Cài đặt theo hướng dẫn trên trang web. Sau khi cài xong, kiểm tra phiên bản: bash

```
node -v  
npm -v
```

- Nếu thấy phiên bản hiển thị (ví dụ: v18.x.x và npm v9.x.x), bạn đã sẵn sàng.

2. **Tạo thư mục dự án:**

- Mở terminal (Command Prompt, PowerShell, hoặc terminal bất kỳ) và chạy: bash

```
mkdir cors-demo-app  
cd cors-demo-app
```

3. Khởi tạo dự án NodeJS:

- Chạy lệnh sau để tạo file package.json: bash

```
npm init -y
```

- Lệnh -y tự động điền các giá trị mặc định. File package.json sẽ là nơi lưu thông tin dự án và các dependencies.

Bước 2: Cài Đặt Các Gói Cần Thiết

1. Cài Express:

- Express là framework giúp xây dựng server nhanh chóng. Chạy: bash

```
npm install express
```

2. Cài CORS:

- Gói cors là middleware để xử lý Cross-Origin Resource Sharing. Chạy: bash

```
npm install cors
```

3. Kiểm tra cài đặt:

- Mở file package.json, bạn sẽ thấy phần "dependencies" đã được cập nhật với express và cors.

Bước 3: Tạo Cấu Trúc Thư Mục và File Cơ Bản

1. Tạo file chính:

- Trong thư mục cors-demo-app, tạo file index.js: bash

```
touch index.js # Trên Linux/Mac  
# Hoặc dùng editor như VSCode để tạo file index.js
```

2. Cấu trúc thư mục (hiện tại đơn giản):

text

```
cors-demo-app/  
├── node_modules/    # Thư mục chứa các gói đã cài  
  
├── package.json      # File cấu hình dự án  
  
└── index.js          # File chính của server
```

Bước 4: Xây Dựng Server với Express và Tích Hợp CORS

1. Mở file `index.js` và thêm đoạn mã sau:

javascript

```
// Import các module cần thiết  
const express = require('express');  
const cors = require('cors');  
  
// Khởi tạo ứng dụng Express  
const app = express();  
  
// Sử dụng middleware CORS  
app.use(cors());  
  
// Middleware để parse JSON (nếu cần gửi dữ liệu dạng JSON từ client)  
app.use(express.json());  
  
// Định nghĩa một route cơ bản  
app.get('/', (req, res) => {  
  res.json({ message: 'Chào mừng đến với CORS Demo App!' });  
});  
  
// Khởi động server  
const PORT = 3000;  
app.listen(PORT, () => {  
  console.log(`Server đang chạy trên cổng ${PORT}`);  
});
```

2. Giải thích mã:

- `const express = require('express');`: Import Express.
- `const cors = require('cors');`: Import gói CORS.
- `app.use(cors());`: Kích hoạt CORS cho tất cả các route, cho phép các yêu cầu từ các nguồn gốc khác.
- `app.get('/')`: Tạo một endpoint GET trả về JSON.

- `app.listen(PORT)`: Khởi động server trên cổng 3000.

Bước 5: Tạo Các Endpoint API Đơn Giản

1. Thêm endpoint mới vào `index.js`:

- Dưới phần route '/', thêm đoạn mã sau:
javascript

```
// Endpoint trả về danh sách dữ liệu giả lập
app.get('/api/data', (req, res) => {
  const data = [
    { id: 1, name: 'Sản phẩm 1' },
    { id: 2, name: 'Sản phẩm 2' },
  ];

  res.json(data);
});

// Endpoint POST để kiểm tra gửi dữ liệu
app.post('/api/submit', (req, res) => {
  const receivedData = req.body;
  res.json({ message: 'Dữ liệu nhận được', data: receivedData });
});
```

2. Giải thích:

- `/api/data`: Trả về một mảng dữ liệu JSON.
- `/api/submit`: Nhận dữ liệu từ client (POST request) và trả về phản hồi.

3. File hoàn chỉnh

javascript

```
// Import các module cần thiết
const express = require('express');
const cors = require('cors');

// Khởi tạo ứng dụng Express
const app = express();

// Cấu hình middleware
app.use(cors()); // Kích hoạt CORS cho tất cả các route
app.use(express.json()); // Parse dữ liệu JSON từ request body

// Định nghĩa các route
// Route GET cơ bản: Trả về thông điệp chào mừng
app.get('/', (req, res) => {
```

```

    res.json({ message: 'Chào mừng đến với CORS Demo App!' });
  });

// Route GET: Trả về danh sách dữ liệu giả lập
app.get('/api/data', (req, res) => {
  const data = [
    { id: 1, name: 'Sản phẩm 1' },
    { id: 2, name: 'Sản phẩm 2' }
  ];

  res.json(data);
});

// Route POST: Nhận dữ liệu từ client và trả về phản hồi
app.post('/api/submit', (req, res) => {
  const receivedData = req.body;
  res.json({
    message: 'Dữ liệu nhận được',
    data: receivedData
  });
});

// Khởi động server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server đang chạy trên cổng ${PORT}`);
});

```

Bước 6: Kiểm Tra Ứng Dụng

1. Khởi động server:

- Trong terminal, tại thư mục cors-demo-app, chạy:
bash

```
node index.js
```

- Bạn sẽ thấy thông báo: Server đang chạy trên cổng 3000.

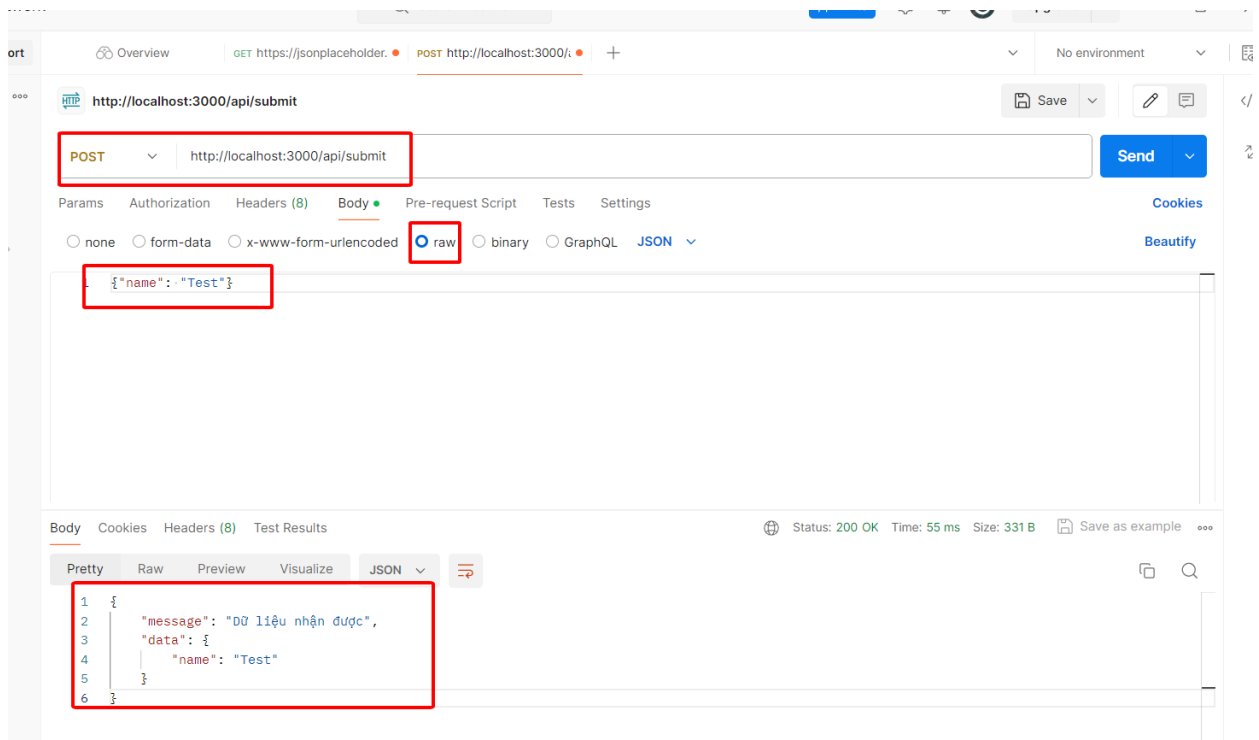
2. Kiểm tra bằng trình duyệt:

- Mở trình duyệt và truy cập: <http://localhost:3000/>.
- Bạn sẽ thấy: {"message": "Chào mừng đến với CORS Demo App!"}.
- Truy cập <http://localhost:3000/api/data> để xem dữ liệu JSON.

3. Kiểm tra CORS bằng Postman:

- Tải [Postman](#) nếu chưa có.

- Gửi yêu cầu GET tới `http://localhost:3000/api/data` → Nhận dữ liệu.
- Gửi yêu cầu POST tới `http://localhost:3000/api/submit` với body JSON (ví dụ: `{"name": "Test"}`) → Nhận phản hồi.



4. Kiểm tra CORS thực tế:

- Tạo một file HTML đơn giản (ví dụ: `test.html`) trên máy tính: html

Preview

```
<!DOCTYPE html>
<html>
<body>
  <button onclick="fetchData()">Lấy dữ liệu</button>
  <script>
    async function fetchData() {
      const response = await
fetch('http://localhost:3000/api/data');
      const data = await response.json();
      console.log(data);
    }
  </script>
</body>
</html>
```

- Mở file này trong trình duyệt → Nhấn nút "Lấy dữ liệu" → Kiểm tra console để xem dữ liệu. Nếu không có lỗi CORS, tức là cấu hình hoạt động đúng.

5. Mở Console Trong Trình Duyệt

Dưới đây là cách mở console trong các trình duyệt phổ biến:

- **Google Chrome:**
 1. Nhấn Ctrl + Shift + J (Windows/Linux) hoặc Cmd + Option + J (Mac).
 2. Hoặc: Nhấp chuột phải trên trang → Chọn "Inspect" (Kiểm tra) → Chuyển sang tab "Console".
- **Mozilla Firefox:**
 1. Nhấn Ctrl + Shift + E (Windows/Linux) hoặc Cmd + Option + E (Mac).
 2. Hoặc: Nhấp chuột phải trên trang → Chọn "Inspect" → Chuyển sang tab "Console".
- **Microsoft Edge:**
 1. Nhấn Ctrl + Shift + J.
 2. Hoặc: Nhấp chuột phải → "Inspect" → Tab "Console".

Lưu Ý và Tùy Chỉnh CORS (Nâng Cao)

- **Cấu hình CORS cụ thể:** Nếu bạn chỉ muốn cho phép một số nguồn gốc (origin) cụ thể, thay `app.use(cors())` bằng:
javascript

```
app.use(cors({
  origin: 'http://example.com', // Chỉ cho phép domain này
  methods: ['GET', 'POST'],     // Chỉ cho phép các phương thức này
}));
```

- **Dừng server:** Nhấn Ctrl + C trong terminal để dừng server khi không dùng.

Kết Luận

Bạn đã tạo thành công một ứng dụng NodeJS với Express và tích hợp CORS! Ứng dụng này có thể:

- Trả về dữ liệu JSON qua các endpoint.
- Nhận yêu cầu từ các nguồn gốc khác mà không gặp lỗi CORS.
- Dễ dàng mở rộng bằng cách thêm các route hoặc logic mới.